



Robert D. Austin

Manager



The Phantom Menace

A long time ago in a company far, far away, I was one of a band of rebels who believed that software development didn't have to be insane. Dissatisfied with perpetual problem-fixing, certain that how we'd been doing things was undisciplined and difficult to improve, we envisioned a better way. We would create a methodology, a repeatable process for developing high-quality software products, one that was helpful rather than onerous. Software development would become a calm, rational process—a kind of engineering.

Methodology products were available commercially, but we opted against these huge, multivolume tomes. They were inaccessible, we reasoned—too cumbersome to use. By specifying too much detail, they restricted developers' abilities and impinged on their discretion. But we studied these methodologies, to understand and avoid their mistakes. We employed experts from renowned universities to guide us. In the end, we produced a thin volume that ran to fewer than 100 pages.

As its opening pages explained, the methodology emphasized *whys*, not *hows*. This guideline suggested what probably needed to be done at each stage of a project and also some ways people had accomplished those things. Far from being restrictive, it left developers free to use their expertise and adapt to the circumstances that were unique to their project. Although it used standard categories for describing project progression—feasibility study, requirements gathering, analysis, design, implementation, testing, customer support—it permitted sophisticated iteration when necessary, back into earlier project stages.

COME THE IMPERIAL ARMIES

With great fanfare, we released this innovation to the development community. Shortly thereafter, the

barrage began. The document was incomplete, said the legions of imperial development troops. It did

Like many others, I became adept at getting the work done while seeming to conform to the oppressive regime—ironic, given my role in its creation.

not explain how to do things. Their leaders were no more satisfied. It did little, they said, to enforce best practice. Furthermore, because it did not enforce a process that was comparable across projects, it did not adequately support organizational learning. To remedy these defects, a committee was formed. Composed not of rebels, but of imperial troops, and commanded by an evil Sith Lord, this committee set immediately to work “filling the holes” in the early document.

After some time, the methodology's next version appeared. Of Death Star proportions, this seven-volume set arrived complete with cross-indexing between volumes. Mandatory training classes would instruct us all on how to interact with this monolith. A software tool was planned to help manage the more than 700 process steps, associated documentation, and milestone reviews that constituted each individual project. As luck would have it, I was by this time a developer on a remote outpost in the manufacturing quadrant. Like many others, I became adept at getting the work done while seeming to conform to the oppressive regime—ironic, given my role in its creation.

Fast-forward now to a conversation I had last week with the CTO of a startup software company. The development process they had used up to that point was at least a little bit insane, he confided. They were spending too much time fixing problems. Their

EDITOR: Roger Pressman • R.S. Pressman & Associates • pressman@rspa.com



Manager

processes were undisciplined and difficult to improve. Success had led to growth, and, as a bigger company, they needed a more formal and mature approach to developing their product.

You can imagine my reaction.

How do you structure a process as complex as software development without killing it? Without

opment processes, but deep down in the organization, you'll find the people who do the real job, managing appearances upward while making software on the front lines.

Here are some characteristics of modern software development processes that I'd suggest are pretty representative of strong development organizations.

That vision we had—of a calm, rational development process, a kind of engineering—it's toast. Gone.

creating that deadly momentum that translates good intentions into a process straitjacket? It is perhaps the fundamental problem of our profession. One might simply argue that we weren't so bright in that faraway galaxy when we created our monster methodology, but I can't write it off that way. I know the folks who created that Frankenstein process specification. I know they were smart—world-class smart, in fact. The question is, how do smart people end up producing such clunky solutions?

In the time since my fateful experience with methodology, I've done a lot of development and visited a lot of development organizations. One thing that's new in the intervening years is the proliferation of companies that sell software commercially and the resulting size of that industry. Software development's mainstream is no longer large companies building systems internally, but smaller companies building systems for sale to other companies and consumers. Not surprisingly, the fierce crucible of market competition has wrought changes in the supporting product-development processes.

And guess what? That vision we had—of a calm, rational development process, a kind of engineering—it's toast. Gone. Obliterated like the planet Alderaan after the Death Star got it.

I don't think I've visited any commercial software companies where the development process is truly calm and rational. What keeps it from happening that way is the pressure to bring new products to market. That's not to say that there is no structure in these environments. What is there, though, doesn't usually have that engineering "look and feel"—and the stability of the structure over time is, truth be told, often missing. Stability can't be there, because changing markets demand changing structures. Some companies cling to handsome fictions about their devel-

♦ *Superstar individuals are the difference between success and failure.* Cruise the code and you'll always find disproportionate evidence of a few strong developers—way disproportion-

ate. Said another way, this software thing is a craft, and stellar craftspeople are the key. It's one of the features we complained most about in our methodology days. We used to talk about taking the art out and making the process more scientific. That's looking increasingly like a fantasy we indulged in from within protected internal IT enclaves.

♦ *Requirements change continuously.* We used to talk about nailing down the requirements up front, engaging users in contracts they would sign off on upon delivery. In market-facing organizations, there is little time for this. It's a fact of business life that important stuff comes up later, and a process that deflects late changes risks irrelevancy.

♦ *Lots of things happen in parallel.* We used to argue that an error caught in testing was a lot more expensive than an error caught much earlier in, say, requirements definition. In the massively parallel development processes of many new organizations, it's challenging to say what "earlier" means, except in a general sense. The best development organizations have worked on making all changes less costly, and on making development a set of concurrent rather than sequential steps. The build cycle organizes development. In a sense, the build cycle has collapsed the stages of our old methodology into a tightly compressed timeframe—sometimes a day (or less). What we're doing is not sequential with support for iteration, like my old version of methodology; rather, it's fundamentally iterative.

THE NEW AGE WAY

One very successful company I know deliberately eschews formal process and even organizational structure. They hire the smartest developers they can find, organize them into very small teams, and



then get out of the way, holding individual developers responsible for the quality of their products (as determined by automated testing routines). Rather than assemble requirement documents, they use their supertalented developers to build overnight prototypes. Even if the prototype is miles off from what a customer wants, the resulting customer reaction is far more informative than the most conscientiously constructed document, and the developer learns what's needed a whole lot more quickly. The development VP there once wrote operating systems at a big company and knows the shortcomings of methodology-intensive ways.

These New Age companies structure their processes only to the extent that it works. If something doesn't work, they either stop doing it or lose in the marketplace. What they evolve in the way of process does sometimes seem to fit with more traditional notions of software methodology. Rapid prototyping is not particularly new. And these companies do adopt more structure as they grow, because they are to some extent overcome by the growing complexity of informal processes (at some point, coordination between different developers gets really hard). In addition, a lot of what they are doing is simple experimentation, some of which will be rejected eventually. But when it comes to structure and the ever-present need to get new product to market, it's a balancing act—a tension that must be managed. It's not about finding an optimal process that can be designed perfectly enough to solve the problem completely.

In this world, the real phantom menace is the seductive idea that you can construct an optimal, engineeringlike process for developing high-quality software products. It's this idea that gets the ball rolling toward the stultifying “filling in” to which my treasured notion of methodological excellence succumbed so long ago. It's what threatens to steal the vitality from agile development organizations, to turn them into stodgy corporate IT department clones. A big part of the problem is with our analogies—the patterns to which we aspire. Is software development really like manufacturing? Is it really like engineering? Maybe we need some new patterns. Last week, I went in search of new patterns. More specifically, I spent the week watching a professional theatre company rehearse. Now, I'm not asking anyone to accept that we should model software develop-

ment processes on theatre rehearsal (not yet, anyway), but let's take the analogy seriously for a moment.

ALL THE WORLD'S A STAGE?

In creating theatre products, participants work through “prototype” performances repeatedly, each time trying to make a better version. No sooner is one version complete than the next suggests itself. You might think theatrical performance is the most repetitive process imaginable because the same words must be said night after night. But as you watch a rehearsal, you realize that this is not correct. There is no presumption that the performance will converge on an ideal. In fact, if variation disappears from the performance, it becomes wooden—downright awful. There is structure, but within the structure is infinite variation as the actors seek to build on each other's actions. Participants are differentially talented and in no way interchangeable. The process is built from the inside out, absorbing internal and external variation and reconceiving the whole. It is intensely iterative and adaptive; it is all about balancing tensions and making quality choices in a given situation. And the process is sufficiently reliable to consistently produce quality outcomes on very firm deadlines (more firm than most software project deadlines—software projects don't sell tickets).

My intention here is not to convince you of the theatre/software analogy, but rather to point out that the patterns, analogies, and language that we

The process is built from the inside out, absorbing internal and external variation and reconceiving the whole.

use severely constrain us as we try to contend with a truly complex process. When we start a conversation about “requirements gathering,” a multitude of potentially limiting presumptions are buried in the phrase. Does the phrase imply sequential rather than parallel project progression? Not necessarily, but in most people's minds it probably does. These presumptions are obstacles to talking about development in any kind of abstract way, something that we must be able to do if we are to discuss its generic structure.



Boston University Department of Electrical & Computer Engineering

If you have significant academic or industrial experience in developing large software systems, and you are committed to improving the practice of software engineering, we want to talk to you about a faculty appointment at Boston University in Computer Systems Engineering. Our graduate program teaches the engineering skills necessary for the effective development of large-scale computer systems in which software provides essential functionality. We teach students to apply engineering principles to the design of a full range of computer products, from embedded systems, to data communication networks, to software products.

Three types of appointments are available in the Department of Electrical and Computer Engineering starting in September 1999:

- Research oriented tenure-track and tenured appointments.
- Non-tenure track positions, which required extensive experience in practicing software engineering.
- Adjunct (part-time) positions for Boston-area experts who are interested in teaching their specialty at the graduate level.

All positions require a commitment to excellence in teaching at the undergraduate and graduate levels.

To learn more about opportunities for a non-tenure track OR adjunct positions, please e-mail: saleh@bu.edu and a faculty member will call to discuss our opportunities. For tenure-track OR tenured appointments, send your Curriculum Vita to: Professor Bahaa E. A. Saleh, Chair, Department of Electrical and Computer Engineering, Boston University, 8 St. Mary's Street, Boston, MA 02215.

**Boston University is an Equal
Opportunity/Affirmative Action**



Manager

What then should we do? What should the CTO I had lunch with do in his organization? One of the best things he and we can do is try to learn how other young, market-facing, rapidly growing companies are doing things. Often these companies are not encumbered by standard patterns and language, simply because they don't have a history of using them. Many are doing a lot of experimenting, which makes it hard to separate what they'll keep doing from what they're just trying and will eventually stop. Also, some of what they're doing might not be relevant to you because of differences in your situation. But plenty nuggets of goodness are there to be had.

A more important reason to keep an eye on these newcomers is that they challenge our established notions about how to do things. Much of what you see might not be directly helpful. But it is usually mind-stretching to see someone break all the rules and still succeed spectacularly.

What else can you do? Maybe go see a play. Something out of the ordinary, maybe Pinter or Pirandello.

And whatever you do—oh, come on, you knew I was going to end this way—may the Force be with you. ❖

Robert D. Austin now professes at the Harvard Business School, but he used to sling code and manage IT operations at a big company. His research focuses on managing software development and other knowledge-intensive activities. He is the author of *Measuring and Managing Performance in Organizations* (Dorset House, New York); raustin@hbs.edu.